DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN
ESCUELA DE INGENIERÍA
PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE

IIC2343 - Arquitectura de Computadores (II/2026)

Documento de preparación para la Actividad 3

Computador básico con saltos

Descripción

El presente documento tiene por objeto preparar al estudiante para la tercera actividad de laboratorio, en la que deberá completar un computador básico capaz de ejecutar saltos. Se recomienda llegar habiendo revisado el diagrama de la CPU, qué recorrido sigue un dato desde que sale de un registro hasta que vuelve a guardarse, y cómo una palabra de bits de la ROM se convierte en las señales que gobiernan ese recorrido.

Alcance de la actividad

- **Se describe:** la arquitectura de `CPU.vhd` (instancias, muxers y cableado interno) y la lógica interna de `CU.vhd`, la unidad de control.
- **Viene entregado:** `Reg.vhd`, `ALU.vhd`, `ROM.vhd`, `RAM.vhd`, `Programmer.vhd`, `Display_Controller.vhd`, `Basys3.vhd` y el ensamblador. Estos archivos no se modifican.
- **Se escribe:** programas en *Assembly* para probar la arquitectura, que el ensamblador traduce y carga en la `ROM`. El trabajo de *hardware* ocurre exclusivamente dentro de la CPU.

Contenidos a tener en cuenta

- **Camino de datos:** qué componentes hay dentro de la CPU y cómo se conectan entre sí.
- **Ciclo de instrucción:** el `PC` direcciona la ROM, la ROM entrega la instrucción y la instrucción determina qué ocurre en ese ciclo.
- **Palabra de control:** cada bit del *opcode* es una señal del circuito; leer la tabla de codificación es leer el cableado.
- **Unidad de control:** lógica **combinacional** que traduce el *opcode* en señales de carga y de selección.
- **Registro de `Status` y saltos:** por qué las *flags* deben guardarse y cómo condicionan la carga del `PC`.
- ***Assembly*:** instrucciones disponibles, *labels* y programas de prueba.

`with/select`, `Reg`, la ALU, las *flags* y el uso de componentes e instancias ya fueron trabajados: este documento solo retoma lo que la CPU agrega.

1. Lo que viene entregado

El proyecto base contiene todos los componentes del computador ya descritos, salvo dos archivos que traen sus asignaciones marcadas con el comentario `-- Corregir`: `CPU.vhd`, donde falta el cableado interno, y `CU.vhd`, donde falta la decodificación. Esos dos archivos son la actividad completa.
Vale la pena abrir los archivos entregados antes del laboratorio, porque contienen información que la actividad no repite. El caso más importante es la ALU: su tabla de operaciones está en el propio `ALU.vhd` y es la que el ensamblador usa al codificar cada instrucción.

`sop`	Resultado	*Flags*
`"000"`	`a + b`	`c` = acarreo de salida
`"001"`	`a - b`	`c` = acarreo de salida, `n` = `a` < `b`
`"010"`	`a and b`	
`"011"`	`a or b`	
`"100"`	`a xor b`	
`"101"`	`not a`	
`"110"`	`a(14 downto 0) & '0'` (izquierda)	`c` = `a(15)`
`"111"`	`'0' & a(15 downto 1)` (derecha)	`c` = `a(0)`

Figura 1: Operaciones de la `ALU` entregada. La *flag* `z` vale `'1'` cuando el resultado es cero, en cualquier operación.

Tres observaciones que se desprenden de esta tabla y que conviene tener presentes:

- Las operaciones `"101"`, `"110"` y `"111"` usan solo el operando `a`, es decir, solo lo que entrega el `Mux A`.
- La *flag* `n` es distinta de cero únicamente en la resta: es la que responde la pregunta *¿fue `a` menor que `b`?*, y por eso las comparaciones se hacen restando.
- La suma con uno de los operandos en cero deja pasar el otro operando sin modificarlo. Ese es el mecanismo con el que se mueven datos en esta CPU.

El otro archivo que conviene revisar es `Reg.vhd`. Es un único componente de 16 bits, y todas las instancias de la CPU salen de él: los dos registros de uso general, el `PC` y el `Status`. En los dos últimos se ocupa solo una parte de los bits y el resto simplemente no se conecta, porque las direcciones son de 12 bits y las *flags* son tres.

Las memorias

Las dos memorias también vienen descritas y se instancian en `Basys3.vhd`, fuera de la CPU. La `ROM` guarda 2^{12} palabras de 36 bits: recibe una dirección de 12 bits y entrega la instrucción completa, sin señal de habilitación de lectura. La `RAM` guarda 2^{12} palabras de 16 bits: recibe una dirección de 12 bits, un dato de entrada y una señal `write`, y entrega de forma permanente el contenido de la dirección que se le está pidiendo.
Esa lectura permanente es la razón por la que una instrucción que opera con un dato de memoria no necesita un ciclo adicional: mientras la dirección esté puesta, el dato ya está disponible en la entrada del `Mux B`. La escritura, en cambio, ocurre en el flanco de subida y solo si `write` está activa.

2. El camino de datos

La CPU no es un componente nuevo que haya que inventar: es la **conexión** de piezas que ya vienen entregadas. El diagrama de la figura 2 es, literalmente, la lista de instancias y de cables que la arquitectura de `CPU.vhd` debe contener: dos registros de uso general, el `PC`, el `Status`, los dos muxers, la ALU y la unidad de control.

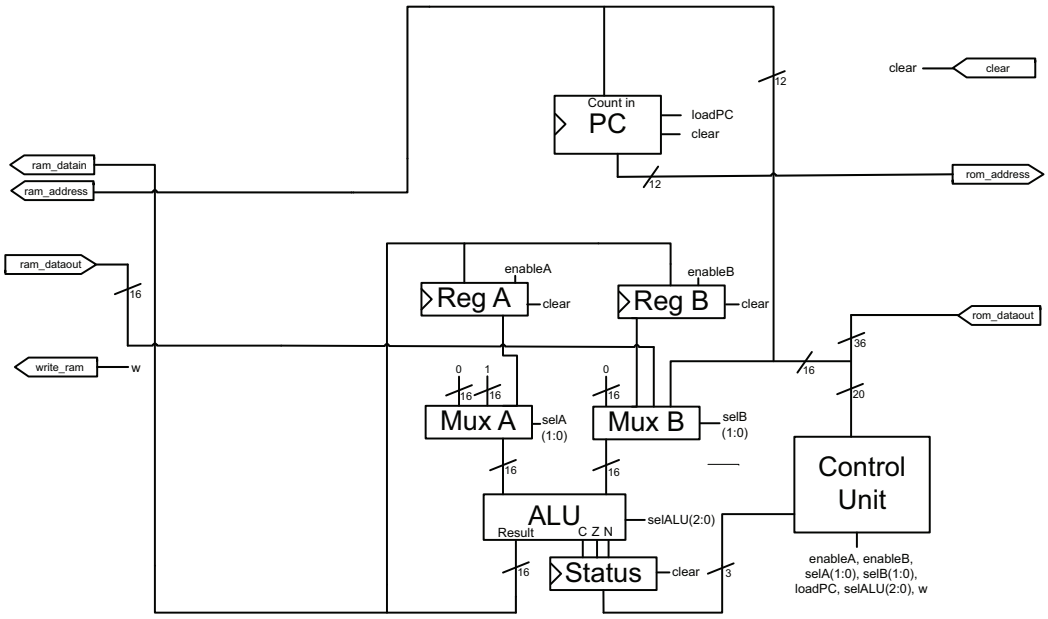


Figura 2: Diagrama interno de la CPU. Conviene identificar en él las cuatro entradas de cada muxer, la ALU y el registro `Status`.

Su entidad ya está definida y es la que `Basys3.vhd` espera encontrar:

```
entity CPU is
    Port (
           clock       : in  STD_LOGIC;
           clear       : in  STD_LOGIC;
           ram_address : out STD_LOGIC_VECTOR (11 downto 0);
           ram_datain  : out STD_LOGIC_VECTOR (15 downto 0);
           ram_dataout : in  STD_LOGIC_VECTOR (15 downto 0);
           ram_write   : out STD_LOGIC;
           rom_address : out STD_LOGIC_VECTOR (11 downto 0);
           rom_dataout : in  STD_LOGIC_VECTOR (35 downto 0);
           dis         : out STD_LOGIC_VECTOR (15 downto 0));
end CPU;
```

Esa entidad es el borde de la figura 2: hacia afuera salen las direcciones y el `display`; hacia adentro queda el cableado que hay que completar.

El recorrido de un dato

Todo dato que se guarda en esta CPU pasa por la ALU. No existe un camino que lleve directamente el contenido de `Reg B` a `Reg A`, ni el literal a un registro: el único cable que llega a la entrada de datos de los registros y a `ram_datain` es la salida de la ALU. En consecuencia, incluso una instrucción que no parece aritmética se ejecuta **operando**, apoyándose en la última observación de la tabla 1.
Los muxers son el punto donde se decide qué participa en la operación. `Mux A` elige entre la salida de `Reg A`, la constante uno y la constante cero; `Mux B`, entre la salida de `Reg B`, el literal de la instrucción, el dato leído de la RAM y la constante cero. Cada uno tiene cuatro entradas de 16 bits y un selector de 2 bits, y se describe dentro de `CPU.vhd` sin necesidad de crear un archivo aparte:

```
-- Ejemplo ilustrativo: la numeracion de las entradas la define quien
-- describe el muxer, y debe ser la misma que use la unidad de control.
with sel_ejemplo select
    salida_ejemplo <= senal_1            when "00",
                      senal_2            when "01",
                      senal_3            when "10",
                      "0000000000000000" when others;
```

La palabra de control no siempre activa un bit de selección. La actividad indica que, en ese caso, el *muxer* puede elegir cualquiera de sus entradas, lo que se resuelve con la alternativa `when others`.

Las direcciones

Además de los datos, la CPU produce dos direcciones de 12 bits. `rom_address` es la salida del `PC`, recortada a los bits que la ROM necesita. `ram_address`, en cambio, no proviene de ningún registro: una instrucción que accede a memoria trae consigo la posición a la que accede, y conviene identificar en el diagrama por qué cable llega ese valor y qué bits de él se conectan.
Por último, la salida `dis` es la concatenación de los 8 bits menos significativos de `Reg A` y de `Reg B`, en ese orden: `dis(15 downto 8)` alimenta los dos *displays* de la izquierda y `dis(7 downto 0)` los de la derecha. Es la única ventana al estado interno de la CPU durante las pruebas.

3. El ciclo de instrucción

En cada flanco de subida del reloj la CPU ejecuta una instrucción completa. La secuencia conceptual es siempre la misma:

1. El `PC` contiene una dirección y la entrega a la ROM por `rom_address`.
2. La ROM entrega por `rom_dataout` la palabra almacenada en esa dirección.
3. Esa palabra se parte en dos: el *opcode*, que va a la unidad de control, y el literal, que va al `Mux B`.
4. La unidad de control traduce el *opcode* en señales; los muxers eligen operandos, la ALU opera y su resultado queda disponible.
5. En el flanco de subida se guarda lo que corresponda (registros, RAM, `Status`) y el `PC` avanza o salta.

Los pasos 1 a 4 son **combinacionales**: ocurren de forma permanente, sin esperar al reloj. Lo único que espera el flanco es el paso 5. Por eso la unidad de control no tiene reloj: no ejecuta pasos, solo describe qué valor tiene cada señal de control mientras esa instrucción esté presente en la salida de la ROM.

La palabra de la ROM

Cada línea de la ROM tiene 36 bits divididos en dos partes: `rom_dataout(35 downto 20)` es el **literal** de 16 bits y `rom_dataout(19 downto 0)` es el ***opcode***, la palabra de control. Separar esas dos partes con rangos es lo primero que hace la CPU con la instrucción que recibe.

El `PC` como contador

El `PC` es una instancia de `Reg`, el mismo componente de la actividad anterior. Su tabla de valores muestra que, en el flanco de subida, puede cargar un valor externo (`load`), incrementarse en uno (`up`) o decrementarse (`down`), y que `clear` lo vuelve a cero sin necesidad de flanco.

Entradas						Salidas
`clock`	`clear`	`load`	`up`	`down`	`datain`	`dataout`
*	1	*	*	*	*	"00...0"
Flanco Subida	0	1	*	*	*	datain
Flanco Subida	0	0	1	*	*	dataout + 1
Flanco Subida	0	0	0	1	*	dataout - 1
*	*	*	*	*	*	dataout

Figura 3: Tabla de valores de `Reg`. Las filas se leen en orden: la primera que aplica es la que manda.

Esas tres primeras filas son todo lo que un `PC` necesita: avanzar a la instrucción siguiente y saltar a otra instrucción son **la misma tabla, leída en dos filas distintas**. Conviene llegar con una respuesta a estas tres preguntas: qué entrada del `PC` corresponde a avanzar, cuál a saltar, y de dónde sale el valor que se carga cuando se salta.
Notar además que `clear` viene del `Programmer`: cuando se carga un programa nuevo por USB, todos los registros vuelven a cero y la ejecución parte desde la primera instrucción sin necesidad de generar otro *bitstream*.

4. La palabra de control

La tabla de codificación del enunciado no es una convención administrativa: es el **cableado** entre la ROM y la CPU. Cada bit del *opcode* corresponde a una señal concreta del diagrama, y el ensamblador se encarga de encender los bits correctos para cada instrucción de *Assembly*. Los bits se agrupan en cuatro familias, y reconocerlas es más útil que memorizar la tabla:

Bits	Familia	Qué hace
`2 downto 0`	operación de la ALU	elige cuál de las ocho operaciones llega al resultado
`5 downto 3`	señales de carga	indican qué elemento guarda ese resultado
`11 downto 6`	selección de muxers	indican qué operandos entran a la ALU
`19 downto 12`	bits de salto	indican si el `PC` carga una dirección en vez de avanzar

Conviene fijarse en cómo la tabla escribe la selección de los muxers: en lugar de entregar directamente el número de la entrada elegida, **dedica un bit distinto a cada alternativa** y enciende el que corresponde. El `Mux A` ocupa así los bits 6, 7 y 8, uno por cada una de sus entradas, y el `Mux B` los bits 9, 10 y 11. El selector de un *muxer*, en cambio, es un número de 2 bits. Pasar de una forma a la otra es parte del trabajo de la unidad de control.

Ejemplo: `MOV A,B`

Tomemos la instrucción `MOV A,B`, que guarda en `A` el valor de `B`. Como no existe un cable directo entre los registros, el valor de `B` tiene que llegar a `A` **pasando por la ALU**: basta con que la ALU sume, con el `Mux A` entregando la constante cero y el `Mux B` entregando la salida de `Reg B`. Con eso, la palabra de control queda así:

Bits	Valor	Por qué
`2 downto 0`	`"000"`	la ALU suma sus dos operandos
`3`	`1`	el resultado se guarda en `Reg A`
`4`	`0`	`Reg B` no se modifica
`5`	`0`	no se escribe en la RAM
`6`	`0`	el `Mux A` no entrega `Reg A`
`7`	`0`	el `Mux A` no entrega el Uno
`8`	`1`	el `Mux A` entrega el Cero
`9`	`1`	el `Mux B` entrega `Reg B`
`10`	`0`	el `Mux B` no entrega el literal
`11`	`0`	el `Mux B` no entrega el dato de la RAM
`19 downto 12`	`0`	no hay salto: el `PC` simplemente avanza
`35 downto 20`	`*`	el literal no participa en esta instrucción

El ejercicio que conviene repetir antes del laboratorio es este mismo, con otras instrucciones de la tabla de *Assembly*: para `MOV A,Lit`, `MOV A,(Dir)`, `MOV (Dir),A`, `ADD A,B`, `INC A` o `CMP A,B`, preguntarse qué entrega cada *muxer*, qué operación hace la ALU y qué elemento guarda el resultado. Varias instrucciones que parecen distintas resultan tener la misma forma, y varias que parecen simples resultan ser una operación aritmética disfrazada.

5. La unidad de control

La unidad de control, `CU.vhd`, es el foco de la actividad. Su entidad viene entregada y su arquitectura viene semi-implementada: las salidas parten asignadas a un valor fijo, marcado con el comentario `-- Corregir`. Es lógica **puramente combinacional**: no tiene reloj, no tiene estados y no recuerda nada. Cada una de sus salidas es una expresión sobre los bits de su entrada, y todas esas expresiones existen al mismo tiempo.
Su entrada es el *opcode* completo, `op_code(19 downto 0)`, y sus salidas son las señales de control que aparecen en el diagrama: los selectores de los dos muxers (`sel_a` y `sel_b`, de 2 bits), las habilitaciones de carga de los registros (`load_a` y `load_b`), la escritura de la RAM (`ram_write`), el selector de operación de la ALU (`sel_op`, de 3 bits) y lo que gobierna la carga del `PC`.

Del bit a la señal

Cuando la tabla asigna un bit completo a una señal de un bit, la traducción es directa. Cuando en cambio hay que mirar varios bits, cada uno de los cuales marca una alternativa distinta, y responder con un selector de 2 bits, hace falta elegir qué valor del selector le corresponde a cada uno de esos bits. Para eso sirve la asignación condicional `when/else`, que es concurrente igual que `with/select`, pero decide según condiciones en vez de según el valor de un único selector:

```
-- Ejemplo ilustrativo: lo minimo para que se ejecute MOV A,B y nada mas.
load_a <= op_code(3);   -- el bit 3 es la carga del registro A

-- Los bits 6, 7 y 8 marcan una entrada cada uno; sel_a es un numero de
-- dos bits, y su correspondencia la fija el muxer descrito en CPU.vhd.
sel_a <= "10" when op_code(8) = '1' else  -- el Mux A entrega el Cero
         "00";                            -- resto de los casos (por definir)
```

Y en `CPU.vhd`, el *muxer* que consume ese selector:

```
-- Ejemplo ilustrativo del Mux A con dos de sus cuatro entradas definidas.
with sel_a select
    mux_a <= reg_a_out          when "00",   -- salida del Reg A
             "0000000000000000" when "10",   -- el Cero
             "0000000000000000" when others; -- alternativa arbitraria
```

Este par de fragmentos alcanza para `MOV A,B` y ninguna otra instrucción. Completar la unidad de control consiste en repetir ese razonamiento para el resto de los bits de la tabla, y en decidir qué valor toma cada selector cuando la palabra de control no enciende ninguno de sus bits.
Notar que no todas las salidas requieren traducción. El selector de operación de la ALU ocupa en la palabra de control los mismos tres bits que la ALU espera recibir, de modo que viaja tal cual; las señales de carga ocupan un bit cada una y también son directas. El trabajo real está en los selectores de los muxers y en la carga del `PC`, que son las únicas salidas que combinan varios bits.
Un punto de atención: cada señal de salida debe describirse en **un solo lugar**. Es tentador escribir una expresión por instrucción de *Assembly*, pero la unidad de control no ve instrucciones: ve bits. La pregunta correcta no es *qué hace* `ADD A,B`, sino *qué valor tiene esta señal en función del opcode*.

6. `Status` y saltos

Sin saltos, un programa es una lista de instrucciones que se ejecuta una vez de principio a fin: no hay condicionales ni ciclos. Los saltos son lo que convierte esa lista en un programa.

Por qué existe el registro `Status`

Las *flags* `c`, `z` y `n` de la tabla 1 describen el resultado que la ALU está calculando **en este instante**. Un salto condicional ocurre en otra instrucción y por lo tanto en otro ciclo: cuando el `JEQ` se está ejecutando, la ALU ya está operando otra cosa y sus *flags* ya no hablan de lo anterior.
Por eso las *flags* se guardan. El registro `Status` es una instancia de `Reg` que conserva las tres *flags* de una instrucción a la siguiente, y se comporta como cualquier registro: carga en el flanco de subida y conserva su valor mientras no se le pida cargar. El salto no exige un `CMP`: solo mira esas *flags* guardadas. `CMP` sirve para tener certeza de qué se comparó (ejecuta la resta sin destruir el registro), pero las *flags* que consulta un salto son las de la última vez que el `Status` se cargó, no necesariamente las de la instrucción anterior.
Vale la pena llegar al laboratorio con una respuesta pensada a: *¿en qué instrucciones debe cargarse el `Status`?*

Qué mira cada salto

La tabla de *Assembly* del enunciado define cada instrucción de salto en términos de las *flags* guardadas en `Status`: `JEQ` exige `Z = 1`, `JNE` exige `Z = 0`, `JLT` exige `N = 1`, y así con las demás; algunas combinan dos condiciones y `JMP` no exige ninguna. La palabra de control, por su parte, dedica un bit distinto a cada tipo de salto, del 12 al 19.
La unidad de control debe entonces producir una única señal de carga del `PC` a partir de dos fuentes: qué tipo de salto pide la instrucción y qué dicen las *flags* guardadas. La forma de esa expresión se deduce leyendo la columna de condiciones de la tabla de *Assembly* junto con los bits 12 al 19 de la palabra de control, y es el ejercicio central de la actividad.
Además, cuando el salto sí se toma, el `PC` debe cargar la dirección de destino. Conviene tener claro de dónde viene esa dirección dentro de la instrucción: es un valor que el ensamblador ya calculó y dejó escrito en la palabra de la ROM.

Comparar sin destruir

La instrucción `CMP A,B` se describe en la tabla como *ejecutar `SUB A,B` sin actualizar el valor de `A`*. Es una buena manera de comprobar si el razonamiento sobre la palabra de control está bien encaminado: la operación de la ALU y la selección de los muxers son idénticas a las de una resta, y la única diferencia está en las señales de carga. Todo lo que `CMP` produce es el efecto sobre las *flags*, y por eso la resta es la operación que responde las comparaciones: su *flag* `n` indica si el primer operando fue menor que el segundo y su *flag* `z` indica si fueron iguales.

7. *Assembly* y ensamblador

El ensamblador toma un archivo de texto con instrucciones de *Assembly*, las traduce a las palabras de 36 bits de la tabla de codificación y las envía por USB a la ROM de la placa; cargar un programa nuevo no requiere volver a sintetizar. Un programa tiene dos secciones: en `DATA:` se declaran variables asociadas a direcciones de la RAM y en `CODE:` van las instrucciones. Un *label* es un nombre seguido de dos puntos que marca una posición del código y es el destino de los saltos. Los literales van en decimal o en hexadecimal con el sufijo `h`, y todo lo que sigue a `//` es un comentario:

```
DATA:

CODE:           // Ejemplo de la forma de un programa

    MOV A,1     // A = 1
    MOV B,1     // B = 1
    CMP A,B     // deja en Status las flags de A - B
    JEQ iguales // salta solo si Z = 1

    NOP         // esta instruccion no deberia ejecutarse

iguales:
    MOV A,AAh   // valor de prueba en A
```

Las variables declaradas en `DATA:` son nombres para direcciones de la RAM, y el ensamblador las reemplaza por el número que les corresponde. Una instrucción puede también usar la dirección directamente, escribiéndola entre paréntesis. Ambas formas producen exactamente la misma palabra de control: la diferencia es de legibilidad del programa, no de *hardware*.
Los programas de prueba de la actividad terminan dejando `AA11` en los *displays*, es decir, `AAh` en los 8 bits bajos de `Reg A` y `11h` en los de `Reg B`. Un programa que salta donde no debe suele terminar en un valor distinto o quedarse pegado, así que el *display* indica que algo falló, pero no dónde: conviene por eso escribir programas propios que ejerciten **una** instrucción a la vez antes de probar un programa completo.
Una forma habitual de comprobar un salto es la del ejemplo anterior: dirigir el caso incorrecto a un *label* que deje en los registros un valor claramente reconocible, de modo que el *display* distinga si el salto se tomó cuando no correspondía.

Observar la ejecución

El computador ejecuta una instrucción por ciclo de reloj y el reloj de la placa es demasiado rápido para seguirlo a simple vista: lo que se ve en los *displays* es el estado final del programa. En `Basys3.vhd` el `Clock_Divider` recibe un selector de velocidad de 2 bits, con cuatro velocidades disponibles, lo que permite bajar el reloj y observar cómo evolucionan los registros instrucción por instrucción. Es la herramienta más útil para entender por qué un programa termina donde no corresponde.
Antes del laboratorio conviene practicar escribiendo, con la tabla de *Assembly* del enunciado a la vista, un programa que recorra un ciclo un número fijo de veces y otro que elija entre dos caminos según una comparación. Ambos son la base de los programas de prueba.

Anexo: simular la CPU en el computador personal

Este anexo es material opcional, pensado únicamente para practicar en casa. En el laboratorio se trabaja exclusivamente con Vivado y el uso de cualquier otro programa está prohibido por el enunciado, de modo que nada de lo que sigue forma parte de la actividad. Su utilidad es otra: permite comprobar en casa, sin la placa a mano, si la propia CPU ejecuta correctamente una instrucción.
La herramienta es `nvc`, un simulador de VHDL libre y liviano que se instala en un par de minutos. Simular requiere un *testbench*, un banco de pruebas que instancia el diseño, le entrega entradas y revisa sus salidas. **Escribirlo no es materia del curso**: va listo en este anexo, para copiar y pegar.

Instalación

Según el sistema operativo:

```
# macOS (con Homebrew)
brew install nvc
# Windows (con winget), o bien el instalador de la pagina de releases
winget install NickGasson.NVC
# Linux: esta empaquetado en Debian, Ubuntu, Fedora y Arch (AUR)
sudo apt install nvc
# Comprobar que quedo instalado
nvc --version
```

Las instrucciones completas y las descargas están en https://github.com/nickg/nvc.

Preparar la carpeta

Los archivos `.vhd` son los del proyecto base publicado en Canvas, `IIC2343_ProyectoBaseACF_3.zip`. Descomprimido, están todos juntos dentro de la carpeta `sources_1/new` del proyecto de Vivado: basta copiar esa carpeta a un lugar cualquiera y agregarle el archivo de simulación de la página siguiente, `tb_CPU.vhd`.
De todos ellos, la simulación solo usa los que están dentro de la CPU: `Basys3.vhd`, las memorias y el `Programmer` quedan fuera, porque el *testbench* ocupa su lugar. No hace falta separarlos, ya que analizar la carpeta completa no causa problemas.

Los tres comandos

Simular es siempre lo mismo: analizar los archivos, elaborar el diseño y ejecutarlo.

```
nvc -a --relaxed *.vhd
nvc -e tb_CPU
nvc -r tb_CPU
```

`-a` analiza (equivale a revisar la sintaxis de cada archivo), `-e` elabora la jerarquía a partir de la unidad de más arriba y `-r` ejecuta la simulación. Los tres pasos pueden encadenarse en una sola línea escribiendo `nvc -a --relaxed *.vhd -e tb_CPU -r`.
El asterisco entrega a `nvc` todos los archivos de la carpeta de una vez, y él resuelve solo en qué orden dependen unos de otros. Si el intérprete de comandos no lo expande, cosa que ocurre en algunas terminales de Windows, se escriben los nombres uno a uno separados por espacios.
La opción `--relaxed` es necesaria porque los archivos entregados contienen expresiones `with/select` que no cubren todos los valores posibles del selector, algo que `nvc` trata como error por omisión y que Vivado acepta sin reclamar. Con `--relaxed` queda como advertencia.

Anexo: el archivo de simulación

No se espera que sepa escribir este archivo ni entenderlo por dentro: los *testbench* no son materia del curso. Se copia y se pega tal cual en un archivo llamado `tb_CPU.vhd`. Lo único que vale la pena reconocer son sus dos constantes, que son las palabras de control de la sección *La palabra de control*: la primera carga `AAh` en `Reg B` desde el literal y la segunda copia `B` en `A`.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity tb_CPU is
end tb_CPU;

architecture Test of tb_CPU is

-- Palabras de control del programa de prueba (bits 19 a 0 de la ROM).
constant MOV_B_LIT : std_logic_vector (19 downto 0) := "00000000010100010000";
constant MOV_A_B   : std_logic_vector (19 downto 0) := "00000000001100001000";
constant NOP       : std_logic_vector (19 downto 0) := "00000000000000000000";
signal clock       : std_logic := '0';
signal clear       : std_logic := '1';
signal ram_address : std_logic_vector (11 downto 0);
signal ram_datain  : std_logic_vector (15 downto 0);
signal ram_dataout : std_logic_vector (15 downto 0) := "0000000000000000";
signal ram_write   : std_logic;
signal rom_address : std_logic_vector (11 downto 0);
signal rom_dataout : std_logic_vector (35 downto 0);
signal dis         : std_logic_vector (15 downto 0);
signal fin         : boolean := false;

begin
clock <= not clock after 5 ns when not fin else '0';   -- reloj de 10 ns
clear <= '0' after 12 ns;                              -- parte con los registros en cero

-- ROM de prueba: dos instrucciones y el resto NOP.
with rom_address select
    rom_dataout <= x"00AA" & MOV_B_LIT when x"000",
                   x"0000" & MOV_A_B   when x"001",
                   x"0000" & NOP       when others;

inst_CPU: entity work.CPU port map(
    clock => clock,             clear       => clear,
    ram_address => ram_address, ram_datain  => ram_datain,
    ram_dataout => ram_dataout, ram_write   => ram_write,
    rom_address => rom_address, rom_dataout => rom_dataout,
    dis => dis );

test : process
    begin
        wait until clear = '0';

        wait until rising_edge(clock);          -- se ejecuta MOV B,AAh
        wait for 1 ns;
        report "MOV B,AAh -> dis = " & to_hstring(dis);
        assert dis = x"00AA" report "FALLO: se esperaba 00AA" severity error;

        wait until rising_edge(clock);          -- se ejecuta MOV A,B
        wait for 1 ns;
        report "MOV A,B   -> dis = " & to_hstring(dis);
        assert dis = x"AAAA" report "FALLO: se esperaba AAAA" severity error;

        fin <= true;
        wait;
    end process;
end Test;
```

Anexo: qué se debería ver

Con la CPU todavía sin completar, las salidas valen cero y la simulación reporta los dos errores:

```
** Note:  16ns+0: MOV B,AAh -> dis = 0000
** Error: 16ns+0: FALLO: se esperaba 00AA
** Note:  26ns+0: MOV A,B   -> dis = 0000
** Error: 26ns+0: FALLO: se esperaba AAAA
```

Cuando el camino de datos y la decodificación de esos bits están bien descritos, desaparecen los errores y quedan solo los dos reportes:

```
** Note: 16ns+0: MOV B,AAh -> dis = 00AA
** Note: 26ns+0: MOV A,B   -> dis = AAAA
```

Los valores se leen igual que en la placa: `dis` son los cuatro dígitos del *display*, con `Reg A` a la izquierda y `Reg B` a la derecha. El primer flanco carga `AAh` en `B` y el segundo lo copia a `A`, de modo que el paso de `00AA` a `AAAA` es exactamente la instrucción `MOV A,B` funcionando.

Ver las señales internas

Cuando el resultado no es el esperado, conviene mirar qué ocurre dentro de la CPU en lugar de adivinar. La opción `--wave` guarda la evolución de todas las señales en un archivo que se abre con un visor de ondas como GTKWave o Surfer:

```
nvc -r --wave=onda.fst tb_CPU
```

Ahí se puede seguir, ciclo a ciclo, qué entrega cada *muxer*, qué señales produce la unidad de control y en qué momento cada registro cambia de valor. Es la versión en pantalla del razonamiento de la sección *El ciclo de instrucción*.

Probar otras instrucciones

Agregar una prueba consiste en escribir la palabra de control de la instrucción que se quiere revisar, siguiendo la tabla de codificación tal como se hizo con `MOV A,B`, sumarla como una alternativa más de la ROM falsa y agregar al proceso el `wait until rising_edge(clock)` y el `assert` correspondientes. Con eso, el mismo archivo sirve para ir comprobando una a una las instrucciones que la CPU debe ejecutar.

Si algo falla antes de simular

Dos mensajes son habituales al empezar. Si al elaborar aparece `cannot find unit WORK.<algo>`, significa que ese archivo no fue analizado: basta agregarlo a la lista del comando `-a`. Si al analizar aparecen errores del tipo `missing choices` o `choices cover only...`, falta la opción `--relaxed`. Ambos son problemas del comando y no de la descripción.
Conviene recordar, en todo caso, que una simulación comprueba la descripción y no el circuito final: lo que la actividad evalúa es el comportamiento del computador en la placa, con el ensamblador cargando los programas de prueba.